Übungsblatt 8: I/O und Systemarchitektur

Mikrocomputertechnik 1

Musterlösung

Aufgabe 1: Zwei Eingabebits und eine Gerätekarte (12 Punkte)

Papierkarte, keine festen Ripes-Adressen: Schalterwort bei `0xE1000000`; sechs Pixel ab `0xE1000080`, Abstand vier Byte, Format `0x00RRGGBB`. Bit 0 und 1 sind aktive High-Eingänge. Maske 3: Ergebnis 3 wählt Gelb `0x00C0C000`, Ergebnis 2 Blau `0x000000C0`, sonst Schwarz.

1. Bestimmen Sie Maskenergebnis und Farbwort für Rohwerte 1, 2, 3 und 7. (4 P)
2. Berechnen Sie Pixeladressen 1 und 4 sowie die erste Adresse hinter dem Pixelbereich. Darf sie als Pixel 6 beschrieben werden? (3 P)
3. Geben Sie die vier Little-Endian-Bytes des gelben Farbworts an. Ist Maskenergebnis 3 schon dieses Farbwort? (2 P)
4. Leiten Sie die Bereichsauswahl für oberste Hexziffer E aus A_{31} bis A_{28} her. Prüfen Sie `0xE1000000` und `0xF1000000`. Wählt diese Bedingung bereits genau das Schalterregister? (3 P)

Musterlösung

1. Rohwert 1 → Maske 1, Schwarz; 2 → 2, Blau; 3 → 3, Gelb; 7 → 3, Gelb. Je Fall 1 P.
2. `0xE1000084`, `0xE1000090`; Grenze `0xE1000098`, kein gültiger Pixel 6. Je gefragter Adressangabe 1 P, Grenzadresse nur mit korrekter Einordnung.
3. Bytes `00 C0 C0 00` (1 P). Nein: Die Maske ist die Eingangsentscheidung, erst danach wird das Farbwort gewählt (1 P).
4. $S_E = A_{31} \wedge A_{30} \wedge A_{29} \wedge \overline{A_{28}}$. Für E eins, für F null. Es werden weitere Adressbits und Zugriffssignale für die Registerauswahl benötigt. Formel 1 P, beide Auswertungen 1 P, Abgrenzung 1 P.

Aufgabe 2: Abfragekosten und ein Vierprobenfilter (12 Punkte)

1. Eine Abfrage braucht bei angenommenen 24 MHz genau 180 Takte, Abfrageperiode 3 ms. Bestimmen Sie Dauer, Zeitanteil und maximale zusätzliche Wartezeit eines bestehen bleibenden Pegels bis zur nächsten Prüfung. (4 P)
2. Ein reines Pegelregister ohne Ereignisspeicherung wird bei 0, 3, 6, 9 ms gelesen; ein Puls liegt vollständig zwischen 4 und 5 ms. Wird er erkannt? Nennen Sie eine Gegenmaßnahme. (2 P)
3. Druck-Freigabe-Erkennung startet freigegeben. Ein separater Filter startet stabil null und übernimmt eine Änderung erst nach **vier** gleichen aufeinanderfolgenden Rohproben. Bestimmen Sie ungefilterte Druckereignisse, jeden Filterzustand und gefilterte Druckereignisse. (4 P)

Probe	1	2	3	4	5	6	7	8	9	10	11	12
Roh	0	1	1	0	1	1	1	1	0	0	0	0
Stabil												

4. Belegt eine Schleife mit 7500 Iterationen bereits eine feste physikalische Entprellzeit? Welche Angaben fehlen? (2 P)

Musterlösung

1. $180/(24 \cdot 10^6) = 7{,}5$ s; Anteil $7{,}5/3000 = 0{,}0025 = 0{,}25\,\%$; zusätzliche Wartezeit bis knapp 3 ms, danach Verarbeitung. Dauer 1 P, Anteil 2 P, Wartezeit 1 P.
2. Nein: Kein Abfragezeitpunkt liegt im Puls (1 P). Schnellere passende Abfrage, dokumentierter Ereignisspeicher oder geeigneter Interruptpfad (1 P).
3. Ungefiltert zwei Druckereignisse an Probe 2 und 5 (1 P). Stabilfolge $0\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 1\ 1\ 1\ 0$ (2 P, anteilig). Ein gefiltertes Druckereignis bei Probe 8 (1 P); Freigabe bei Probe 12.
4. Nein (0,5 P). Mindestens Taktfrequenz, ausgeführte Maschinenbefehle, Taktkosten und gegebenenfalls Unterbrechungen fehlen (1,5 P).

Aufgabe 3: Andere Trap-Ursachen und Einsprungadressen (12 Punkte)

RV32-Privilegienmodell mit M und U, keine Delegation oder Verschachtelung. Angegebene mtvec-Werte werden unterstützt. Für diese Aufgabe: Interruptcode 3 = Machine Software, Code 7 = Machine Timer; Exceptioncode 2 = Illegal Instruction.

1. Decodieren Sie `mcause=0x80000007`, `0x80000003` und `0x00000002`. (3 P)
2. Bestimmen Sie die Handlerziele und erklären Sie die unteren mtvec-Bits. (5 P)

mtvec	mcause	Ziel
`0x200`	`0x80000007`	
`0x201`	`0x80000007`	
`0x201`	`0x80000003`	
`0x201`	`0x00000002`	

3. mepc = `0x00400230` in drei unabhängigen Fällen: Interrupt, vollständig bearbeiteter 32-Bit-ecall, behobener Fehler mit gewünschtem Wiederholungsversuch. Nennen und begründen Sie die Fortsetzung jeweils. Warum ist pauschales Addieren von vier falsch? (4 P)

Musterlösung

1. Timer-Interrupt, Software-Interrupt, Illegal-Instruction-Exception. Je 1 P. Bit 31 trennt Interrupt und Exception; der Rest enthält den Code.
2. Ziele `0x200`, `0x21C`, `0x20C`, `0x200`. Je 1 P. mtvec-MODE 00 bedeutet Direct; 01 Vectored. Nur Interrupts im Vectored-Modus nutzen BASE + viermal Code (1 P). BASE bleibt hier `0x200`.
3. Interrupt: `0x00400230`, gespeicherte Fortsetzung. Bearbeiteter ecall: `0x00400234`, Anfrage überspringen. Wiederholungsfall: `0x00400230`, dieselbe Instruktion erneut versuchen. Je 1 P. Pauschales Erhöhen würde gegebenenfalls notwendige Arbeit überspringen (1 P).

Aufgabe 4: Freigabe löschen und aus U zurückkehren (10 Punkte)

RV32-Statusfelder: MIE Bit 3, MPIE Bit 7, MPP Bits 12:11; U = 0, M = 3.

1. Unabhängiger Bitfall: mstatus = `0x00000088`. Welches Bit wird durch den Code gelöscht, welches Wort entsteht und welches relevante Bit bleibt erhalten? (3 P)

```
li    t1, 8
csrrc x0, mstatus, t1
```

2. Im gerade laufenden M-Programm sind Quelle, Zielprivileg und Prioritäten geklärt. Entscheiden Sie für $(P, E, MIE) = (0,1,1), (1,1,0), (1,0,1), (1,1,1)$ über die Interruptannahme. (2 P)
3. Verhindert MIE = 0 einen synchronen ecall? (1 P)
4. Ein U-Programm löst ecall aus. Vorher: aktueller Modus U, MIE = 0, MPIE = 1, MPP = U. Handler verändert diese Felder nicht eigens und kehrt mit mret zurück. Bestimmen Sie die vier Größen im Handler und danach. (4 P)

Musterlösung

1. Bit 3 (MIE) wird gelöscht; Ergebnis `0x00000080`; MPIE in Bit 7 bleibt eins. Je 1 P. csrrc löscht nur maskierte schreibbare Bits.
2. Nein, nein, nein, ja. Je 0,5 P; es gilt hier $P \wedge E \wedge MIE$.
3. Nein: Die maskierbare Interruptfreigabe unterdrückt keine synchrone Environment-Call-Exception. 1 P.

Größe	Im Handler	Nach mret
aktueller Modus	M	U
MIE	0	0
MPIE	0	1
MPP	U	U

4. Je Feldwert 0,5 P. Beim Eintritt wird das frühere MIE = 0 nach MPIE kopiert; MPP speichert den tatsächlichen Herkunftsmodus U. mret setzt MPIE anschließend auf eins.

Aufgabe 5: Zwei unterbrochene Werte erhalten (8 Punkte)

Gültiger Handlerstack, sp = `0xB000`, t1 = −12, a2 = 44. Keine Verschachtelung. Der Leaf-Ausschnitt verändert nur diese Datenregister und sp.

```
addi sp, sp, -16
sw   t1, 0(sp)
sw   a2, 4(sp)
li   t1, 200
addi a2, t1, 1
lw   a2, 4(sp)
lw   t1, 0(sp)
addi sp, sp, 16
mret
```

1. Geben Sie neuen sp, Framebereich und beide Sicherungsadressen an. (3 P)
2. Bestimmen Sie Endwerte t1, a2, sp. Warum entbindet caller-saved den Handler nicht von der Erhaltung? (3 P)
3. Ein anderer Handler ruft einen ABI-konformen Helfer und verändert selbst s3 und s4. Berechnen Sie Nutzbedarf und kleinsten 16-Byte-ausgerichteten Frame allein für ra, t0–t6, a0–a7, s3 und s4. (2 P)

Musterlösung

1. sp = `0xAFF0`, Frame `0xAFF0`–`0xAFFF`; Slots `0xAFF0` und `0xAFF4`. sp und Bereich zusammen 1 P, Slots je 1 P.
2. t1 = −12, a2 = 44, sp = `0xB000` (zusammen 1 P). Der asynchron unterbrochene Code hat keinen normalen Aufruf vorbereitet und konnte diese Werte nicht zuvor selbst schützen. Die Trap-CSRs sind kein Integerregister-Backup (2 P).
3. $1 + 7 + 8 + 2 = 18$ Wörter = 72 Byte; Frame 80 Byte mit acht Byte Padding. Nutzbedarf und Frame je 1 P. Bereits enthaltene t-/a-Register werden nicht doppelt gezählt.

Aufgabe 6: Adresse als Zahl oder Text (6 Punkte)

1. Eine Venus-Lehradresse `0x10010040` zeigt auf einen nullterminierten Text. Vor ecall stehen a0 = 1 und a1 = diese Adresse. Was wird angefragt? Wie korrigieren Sie zur beabsichtigten Textausgabe? (2 P)
2. Warum sichert ecall keinen jal-Link in ra? Ist s2 allein wegen seines ABI-Namens über jeden beliebigen Umgebungsdienst garantiert erhalten? (2 P)
3. Eine fiktive Umgebung darf alle t- und a-Register verändern, erhält aber ausdrücklich s2, sp und benutzten Speicher. Nennen Sie zwei zulässige Möglichkeiten, einen Zähler über den Dienst hinweg zu behalten. (2 P)

Musterlösung

1. Dienst 1 druckt die Adresse als Ganzzahl, hier 268501056 (1 P). Für Text a0 = 4 setzen und dieselbe Adresse in a1 übergeben (1 P).
2. ecall ist eine Environment-Anfrage mit umgebungsspezifischer Fortsetzung, kein normaler Funktionsaufruf. Erhaltung folgt dem Dienstvertrag und nicht allein dem Namen s2. Je 1 P.
3. Zähler in dem ausdrücklich erhaltenen s2 halten; oder auf gültigem ausgerichtetem Stack sichern und danach laden, weil sp und Speicher zugesichert sind. Je begründeter Strategie 1 P.

Aufgabe 7: Eine Zweibit-Anzeige in Ripes

1. Konfigurieren Sie mindestens zwei aktive High-Schalter und mindestens zwei LED-Pixel. Übernehmen Sie die tatsächlichen Exports SWITCHES_0_BASE und LED_MATRIX_0_BASE.
2. Implementieren Sie die Farbauswahl aus Aufgabe 1 für **Pixel 1**. Prüfen Sie Rohwerte 1, 2, 3, 7 und danach 0; andere Pixel sollen unverändert bleiben.
3. Protokollieren Sie Rohwert, Maske, tatsächliche Store-Adresse, Farbwort und sichtbare Anzeige. Weshalb muss auch Schwarz ausdrücklich geschrieben werden?
4. Vergleichen Sie danach mit `beispiele/lab_08_zweibit_anzeige.s`. Notieren Sie Ripes-Version und Modell; die Papieradressen gelten nur bei passender Gerätekarte.

Musterlösung

Masken 1, 2, 3, 3, 0 liefern Schwarz, Blau, Gelb, Gelb, Schwarz. Jeder Store trifft die tatsächliche LED-Basis plus vier. Nur Pixel 1 wird verändert. Der letzte Null-Store löscht den zuvor gelben Pixel; bloßes Überspringen eines Stores würde die alte Farbe behalten.

Die Referenz benötigt die zwei echten exportierten Plattformkonstanten. Sie behauptet weder feste universelle MMIO-Adressen noch einen Interruptpfad. Sichtbare Gerätewirkung ist im eigenen Ripes-Lauf zu prüfen.

Aufgabe 8: Einen korrigierten Messwert in Venus ausgeben

1. Schreiben Sie ein vollständiges Programm, das zuerst `Messwert:` als Text und dann $3n - 4$ als signed Ganzzahl mit Zeilenumbruch ausgibt. n liegt als Datenwort vor; testen Sie 17, −3 und 0.
2. Berechnen Sie den dreifachen Wert mit Schieben und Addition. Laden Sie n erst **nach** dem Präfixdienst neu; verlassen Sie sich nicht auf undokumentierte Erhaltung temporärer Register.
3. Protokollieren Sie vor jedem ecall Dienstkennung und Zahl beziehungsweise Textadresse. Venus-Kursdienste: 4 Text, 1 Ganzzahl, 10 Ende.
4. Vergleichen Sie anschließend mit `beispiele/lab_08_messwertausgabe.s`. Warum demonstriert das keinen selbst geschriebenen privilegierten Handler?

Musterlösung

Erwartete Zeilen: `Messwert: 47`, `Messwert: -13`, `Messwert: -4`. Nach dem Textdienst lädt die Referenz n erneut, bildet $2n + n - 4$ und übergibt das Ergebnis in a1 an Dienst 1. Für Präfix und Zeilenumbruch steht jeweils die Adresse einer nullterminierten Zeichenfolge in a1.

Venus übernimmt die Dienstbehandlung intern. Die Funktion eines selbst programmierten M-Mode-Handlers wird damit nicht nachgewiesen; die Ausgaben sind Vorhersagen, keine bereits protokollierten Läufe.